PROJECT TECHNICAL DOCUMENTATION & SPECIFICATIONS

NodeBeacon: Infrastructure Monitoring & Telemetry Platform

A Full-Stack Server Health Surveillance, Incident Alerting & Remote Diagnostic System

Author / Developer: Neeraj	**Documentation Date:** October 20, 2026
Architecture: 3-Tier Distributed (Agent / REST API / SPA)	**Stack:** React 19, Django 5.2, MySQL 8, Python 3.11
Target Operating Systems: Windows 10/11, Windows Server, Linux	**Security Model:** JWT SimpleJWT, Google OAuth 2.0, SHA-256 Tokens

1. Executive Summary & Problem Statement

Modern digital services rely on continuous server availability. Traditional enterprise monitoring platforms (such as Datadog, Splunk, or New Relic) are often heavyweight, complex, costly, and demand high resident memory overhead on monitored nodes. **NodeBeacon** was engineered to solve this challenge by providing an autonomous, ultra-lightweight, and developer-friendly infrastructure monitoring suite.

Core Pillars of NodeBeacon:

- **Autonomous Telemetry Agent:** A standalone compiled Windows executable (<20 MB RAM footprint) requiring zero client configuration, capturing CPU, RAM, Disk, Network throughput, and OS uptime.
- **Automated Incident Engine:** Continuously evaluates server heartbeats and resource threshold violations; triggers automated alerts via responsive HTML emails, Telegram bot notifications, and desktop notifications.
- **Dual Authentication & Google SSO:** Implements secure JSON Web Token (JWT) sessions, granular Role-Based Access Control (RBAC), and Google OAuth 2.0 Single Sign-On with cryptographically verified state tokens.
- **Dynamic Audit PDF Compilation:** Generates multi-page executive performance audit reports with high-resolution Matplotlib trend charts and ReportLab vector tables directly in memory.
- **Laptop Session Physical Security:** Native Windows Terminal Services (Wtsapi32.dll) event listener that alerts administrators whenever a monitored computer session is unlocked.

2. Technical Specifications Used

Tier / Layer	Technology / Library	Version	Role & Specification in NodeBeacon
Frontend UI	React	19.2.7	Core component rendering, virtual DOM state synchronization
Build Tool	Vite	8.1.1	Blazing-fast ESM dev server, bundling, hot module replacement
Client Routing	React Router DOM	7.18.1	Declarative routing, protected route boundaries, history management
CSS & Styling	Tailwind CSS	4.3.3	Utility-first design system with persistent dark/light theme switching
Data Viz	Recharts	3.9.2	Interactive SVG time-series charts for CPU, RAM, Disk & Network
Micro-Animations	Framer Motion	12.42.2	Fluid page tab transitions and status notification badges
Icons	Lucide React	1.25.0	Clean, modern SVG icons for system dashboards and server states
HTTP Client	Axios	1.18.1	Asynchronous REST requests, JWT bearer authorization interceptor
Backend Framework	Django (Python)	5.2+	Core Web Framework, ORM layer, settings, email dispatch, admin
REST API	Django REST Framework	3.17.1	Class-based API views, model serializers, rate limit throttles
Authentication	DRF SimpleJWT	5.5.1	HMAC SHA-256 signed access (short) and refresh (long) JWT tokens
SSO Integration	Google OAuth 2.0	RFC 6749	Single Sign-On with cryptographically verified HMAC state payloads
Database Engine	MySQL Server	8.x	Relational storage with utf8mb4 collation and foreign key cascades
Database Driver	mysqlclient	2.2.0+	C-optimized direct database connector for high throughput
PDF & Charting	ReportLab & Matplotlib	5.0 / 3.11	In-memory dual-canvas PDF compilation and 4-quadrant telemetry plots

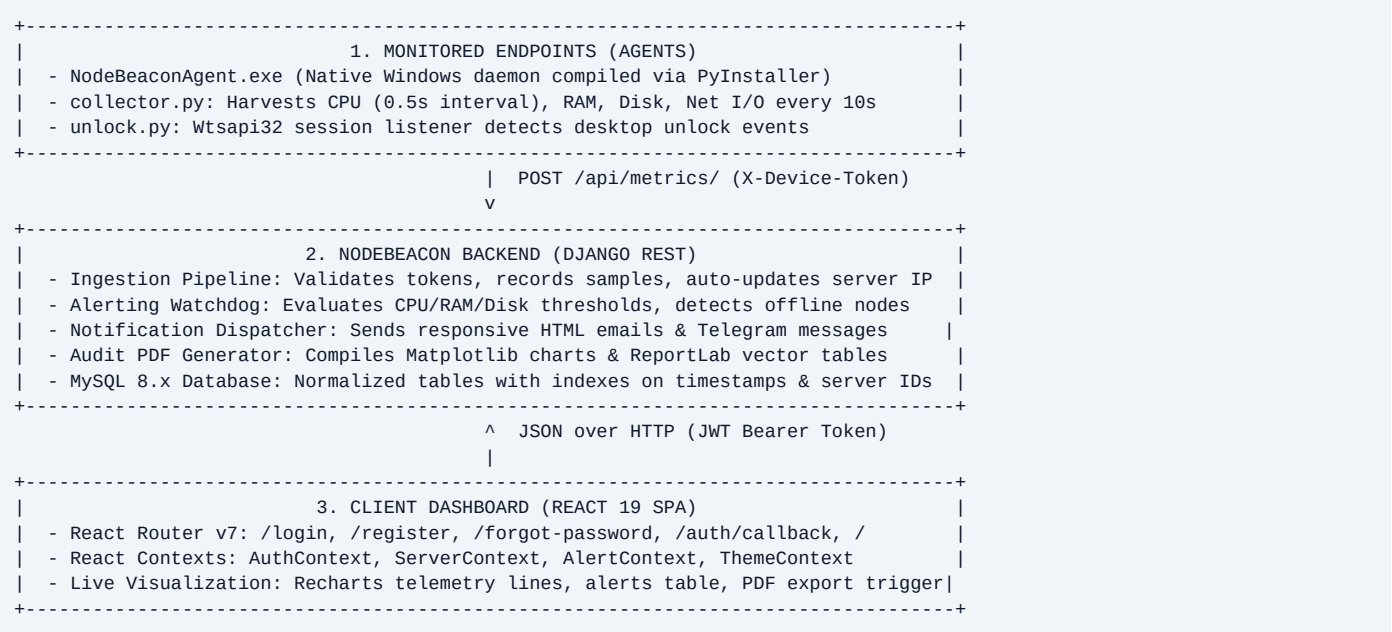
Monitoring Agent	Python Native Binary	3.11	Autonomous telemetry daemon running every 10s on endpoints
OS Metrics	psutil	7.2.2	Harvests CPU (0.5s slice), virtual RAM, disk partition, net I/O, boot time
Windows API	pywin32 & Wtsapi32	>=306	Windows Terminal Services hooks for laptop unlock detection
Binary Packaging	PyInstaller	6.21.0	Packages agent into standalone zero-dependency Windows executable

2.1 System & Hardware Requirements

Parameter	Host / Cloud Server Specification	Monitored Endpoint (Agent) Specification
Processor (CPU)	Dual-core 2.0 GHz or higher (x86_64)	Any single or multi-core x86/x64 processor (<0.5% load)
Memory (RAM)	Min 2 GB (4 GB recommended for PDF/DB)	<25 MB RAM consumption during execution
Disk Storage	5 GB free space for OS, DB & log retention	35 MB free disk space for executable & buffers
Operating System	Windows 10/11, Windows Server, Linux (Ubuntu)	Windows 10, Windows 11, Windows Server (Linux compatible)
Network Ports	8000 (Django API), 5173 (Vite), 3306 (MySQL), 587 (SMTP)	Outbound HTTP/HTTPS to port 8000 (No inbound ports needed)

3. System Architecture & High-Level Design

NodeBeacon is designed around an asynchronous, non-blocking telemetry model. Communication between tiers occurs over standard RESTful HTTP with strict separation of concerns.



4. Detailed Working of Subsystems

4.1 Authentication & Security Subsystem

NodeBeacon provides an enterprise-ready security perimeter supporting dual authentication models for users and machines:

- **JWT Token Lifecycle:** Standard login (`/api/auth/login/`) validates salted password hashes and delivers an access token (5-minute TTL) and refresh token (24-hour TTL). The React frontend stores tokens securely in browser local storage and injects them into Axios request headers via HTTP interceptors. Silent token refresh (`/api/auth/refresh/`) guarantees seamless user experiences.
- **Google OAuth 2.0 Single Sign-On:** Built on RFC 6749. When the user selects 'Sign in with Google', the backend creates a cryptographically signed HMAC SHA-256 `state` token containing a timestamp and target redirect URI. Upon Google callback, the backend validates the state signature, performs server-to-server code exchange with Google token endpoints, verifies identity claims, and links or creates the user record.
- **Role-Based Access Control (RBAC):** Accounts are segregated into *Individual* (personal server fleets) and *Organization* (shared enterprise fleets). Organizations support tiered member roles: *Administrator* (full fleet creation, configuration, and API reset permissions), *Operator* (alert acknowledgement and operational restarts), and *Viewer* (read-only telemetry metrics inspection).
- **Device Credential Isolation:** Machine agents do not use user credentials. Instead, each server entity has a 48-character API key and an enrolled 32-byte `device_token`. Only SHA-256 hashes (`device_token_hash`) are persisted in the MySQL database, ensuring compromise of the database never leaks cleartext machine credentials.

4.2 Endpoint Monitoring Agent Engine

The endpoint agent (`agent/`) is written in Python 3.11 and packaged into a standalone Windows binary (`NodeBeaconAgent.exe`) via PyInstaller. It functions autonomously with the following lifecycle:

1. **Zero-Friction One-Click Enrollment:** When a user registers a server, they click **Download Agent**. The backend bundles the executable and an ephemeral `enrollment.json` into a ZIP archive. The JSON file contains a one-time 32-byte token valid for 30 minutes. Upon initial launch, the agent calls `/api/servers/agents/register/`, consumes the token atomically with database row locks (`select_for_update`), and receives its permanent device token and telemetry configuration.
2. **High-Precision Metric Sampling:** Every 10 seconds, `collector.py` executes: CPU utilization via `psutil.cpu_percent(interval=0.5)` to measure real compute load rather than an instantaneous spike; Virtual memory via `psutil.virtual_memory().percent`; Disk utilization targeting `C:\`; Network I/O byte counters (`bytes_sent`, `bytes_recv`); System boot time and uptime; and OS kernel build analysis to distinguish Windows 10 vs. Windows 11.

3. **Exponential Backoff & Network Resilience:** Telemetry is posted to `/api/metrics/`. If the backend becomes unreachable, `runtime.py` catches the network timeout, sets local agent status to `Reconnecting`, and throttles retry attempts using an exponential delay sequence (`5s -> 10s -> 30s -> 60s`). When connectivity is restored, normal 10-second sampling resumes without crashing.
4. **Windows Session Physical Unlock Detection:** Implemented in `unlock.py`. The agent spins up a dedicated background thread with a Win32 message loop registered to `Wtsapi32.dll!WTSRegisterSessionNotification`. When an authenticated user unlocks the computer screen (`WTS_SESSION_UNLOCK = 0x8`), the event is captured, assigned an idempotent UUIDv4, filtered against a 3-second deduplication window, and transmitted to `/api/servers/agents/unlocked/` to notify the administrator.

4.3 Ingestion & Automated Alerting Engine

The backend ingestion pipeline processes thousands of metrics with instant threshold evaluation:

- **Telemetry Ingestion (`MetricsListCreateView`):** Validates `X-Device-Token`, saves the sample to `metrics_metric`, updates server status to `Online`, updates `last_seen` timestamp, and detects dynamic client IP changes through `HTTP_X_FORWARDED_FOR`.
- **Real-Time Threshold Evaluation (`evaluate_metrics_alerts`):** The new metric is checked against the user's configured limits: CPU limit (e.g. >90%), Memory limit (e.g. >90%), and Disk limit (e.g. >95%). If breached, a new `Alert` is created with `severity='Critical'` and `status='Active'`. When metrics return to safe ranges in future samples, the alert is automatically resolved (`status='Resolved'`).
- **Offline Heartbeat Watchdog (`check_and_trigger_offline_servers`):** If a server stops transmitting metrics for longer than `heartbeat_timeout` (default 60s), the server status flips to `Offline` and a critical 'Server Offline' alert is broadcast.
- **Multi-Channel Alert Dispatch:** Alerts are immediately formatted into a responsive HTML email with server metadata and dispatched via SMTP. If the user paired their Telegram bot using the deep-linking verification code (`users_telegrampairing`), an instant Telegram alert message is also delivered.

4.4 Dynamic Audit & PDF Reporting Engine

Implemented in `backend/metrics/views.py` (`ExportPDFReportView`). Administrators can download complete infrastructure health audits at any time:

- **In-Memory Visualization:** Uses Matplotlib with the non-GUI `Agg` backend to construct a 4-quadrant subplot (CPU Load Trend, Memory Usage, Disk Capacity, and Network Activity KB/s) directly into a BytesIO memory buffer.
- **ReportLab Vector Layout:** Assembles an executive report containing cover branding, KPI metric summary tiles (Total Servers, Online Ratio, Fleet Uptime %, Average Loads), server inventory table with colored status badges, embedded telemetry trend charts, and incident resolution audit tables.
- **Two-Pass Dynamic Pagination:** Employs a custom `NumberedCanvas` to calculate total pages and output running header and footer banners (`Page X of Y`).

4.5 Frontend React SPA Architecture

The React 19 web application is structured for fast response times, smooth navigation, and clear separation of state:

- **Declarative Routing (React Router v7):** Standard URL routing manages views (`/login`, `/register`, `/forgot-password`, `/auth/callback`, and protected `/`). Browser history buttons (Back/Forward) function naturally, and OAuth callbacks are parsed automatically.
- **State Management via React Contexts:** `AuthContext` provides authenticated user profile and tokens; `ServerContext` supplies cached server lists and mutation hooks; `AlertContext` manages live unread notifications; and `ThemeContext` coordinates persistent dark/light theme switching via browser `localStorage`.
- **Interactive Dashboard & Live Charts:** The main view offers a fleet-level summary card grid, live Recharts line charts with SVG tooltips, and server details drill-downs displaying real-time telemetry gauges.
- **Desktop Web Push Notifications:** Uses the HTML5 `Notification` API to alert administrators of critical incidents even when the browser tab is minimized in the background.

5. Database Schema & Relational Design

NodeBeacon utilizes MySQL 8.x with InnoDB storage engine for ACID transaction guarantees, strict foreign keys, and optimized composite B-tree indexes.

Table Name	Primary Key & Relationships	Key Fields & Types	Indexes & Optimization Purpose
users_user	id (PK) organization_id (FK)	email (UK), role, cpu_threshold, memory_threshold, disk_threshold, heartbeat_timeout, telegram_chat_id, google_id	Indexed on email and google_id for sub-millisecond login lookups.
users_organization	id (PK) owner_id (FK to User)	organization_name, alert_email, alert_recipients (Text), created_at	Normalizes multi-tenant company accounts and shared notification lists.
servers_server	id (PK) owner_id (FK) organization_id (FK)	name, hostname, operating_system, ip_address, api_key (UK), status, device_id (UUID), device_token_hash, last_seen	Composite indexes: (owner, -created_at) and (device_token_hash).
metrics_metric	id (PK) server_id (FK CASCADE)	cpu_usage (Float), memory_usage (Float), disk_usage (Float), network_in (BigInt), network_out (BigInt), uptime, timestamp	High-write append table. Composite index: (server, -timestamp) for fast chart telemetry queries.
alerts_alert	id (PK) server_id (FK) metric_id (FK Null)	title, description (Text), severity ('Critical'/'Warning'), status ('Active'/'Resolved'), created_at, resolved_at	Composite indexes: (server, status) and (status, -created_at).
servers_agentenrollment	id (PK) server_id (FK CASCADE)	token_hash (UK), expires_at (DateTime), used_at (DateTime Null), created_at	Enforces single-use 30-minute agent installation handshakes with row locks.
servers_unlocknotification	id (PK) server_id (FK CASCADE)	event_id (UUID UK), delivery_status ('Pending'/'Sent'), created_at, attempted_at, delivered_at	Composite index: (server, -created_at). UUID UK guarantees delivery idempotency.

6. REST API Endpoints Specification

HTTP Method	API Endpoint URI	Authentication	Functional Description
POST	/api/auth/register/	Public	Registers a new administrator account
POST	/api/auth/login/	Public	Authenticates credentials and returns JWT Access & Refresh pair
GET	/api/auth/google/url/	Public	Returns Google OAuth 2.0 authorization URL with signed state
POST	/api/auth/google/	Public	Exchanges Google authorization code for JWT user session
POST	/api/auth/refresh/	Public	Renews expired access token using valid refresh token
GET / PATCH	/api/auth/profile/	JWT Bearer	Fetches or updates user alert thresholds and contact info
GET / POST	/api/servers/	JWT Bearer	Lists all fleet servers or registers a new server entity
GET / DELETE	/api/servers/<id>/	JWT Bearer	Inspects or unregisters a specific server node
POST	/api/servers/<id>/agent-download/	JWT Bearer	Generates customized one-click Windows agent ZIP package
POST	/api/servers/agents/register/	Public (Token)	Agent enrollment handshake; exchanges one-time token for device token
GET / PATCH	/api/servers/agents/config/	Device Token	Fetches or patches remote polling interval and backend target URLs
POST	/api/servers/agents/heartbeat/	Device Token	Endpoint process heartbeat ('Connected', 'Reconnecting', 'Stopped')
POST	/api/servers/agents/unlocked/	Device Token	Delivers idempotent Windows session unlock notification event
POST	/api/metrics/	Device Token / Key	Ingests compute telemetry and triggers threshold alert checks
GET	/api/metrics/	JWT Bearer	Retrieves latest telemetry metrics across all owned servers
GET	/api/servers/<id>/metrics/	JWT Bearer	Retrieves historical time-series metrics for server charts
GET	/api/metrics/export-pdf/	JWT Bearer	Downloads dynamic PDF performance audit report with charts
GET / PATCH	/api/alerts/	JWT Bearer	Lists active and resolved alerts or marks incident resolved

7. End-to-End Execution Workflow

Phase	Action / Trigger	System Internal Mechanics
Phase 1: Onboarding	User registers or signs in with Google SSO.	Backend creates User record, establishes JWT session tokens, and loads default threshold settings (90% CPU, 90% RAM, 95% Disk).
Phase 2: Fleet Provisioning	Admin adds a server and downloads the agent installer.	Backend generates unique Server entity, issues an ephemeral 30-minute enrollment token, and bundles NodeBeaconAgent.exe with enrollment.json.
Phase 3: Agent Activation	Admin extracts and launches the executable on the target server.	Agent exchanges enrollment token for permanent device_token. Agent starts background collection loop and session unlock hooks.
Phase 4: Telemetry Streaming	Agent posts metrics every 10s via HTTPS.	Backend writes metric to MySQL, updates server status to 'Online', refreshes last seen time, and pushes latest points to dashboard charts.
Phase 5: Incident Resolution	CPU utilization exceeds threshold or heartbeat is missed.	Backend creates Active Critical Alert, dispatches responsive HTML email and Telegram notification. Resolves alert automatically once compute load stabilizes.

8. Key Advantages & Presentation Highlights

Summary of Distinct Engineering Highlights:

1. **Zero Configuration Onboarding:** Monitored servers require no Python installation, no manual credential typing, and no complex service configuration.
2. **True Multi-Channel Alerting:** Instantaneous failover alerts delivered via SMTP Email, Telegram Bot deep links, and desktop notifications.
3. **Dual Authentication Boundaries:** Human users authenticate with JWT and Google OAuth SSO, while server machines communicate strictly via SHA-256 hashed device tokens.
4. **Low Resource Consumption:** The native monitoring daemon consumes less than 20 MB of resident RAM and under 0.5% CPU overhead.
5. **Dynamic In-Memory PDF Reporting:** Generates vector audit reports with dynamic page counts and 4-quadrant telemetry plots without saving temporary files to disk.
6. **Persistent Visual Themes:** Sleek dark/light theme switching with smooth transitions and persistent browser preferences.